

COURSEWORK_SPEC

Coursework Specification: Campus Delivery Agent

Course: Foundations of AI (COMP1037)

Institution: University of Nottingham Ningbo China

Duration: 3 weeks

Submission: Individual, via Moodle (auto-graded on Gradescope)

Format: Single Python file `campus_delivery.py`

Deadline: 4pm, 11 May 2026

Total Points: 100

Table of Contents

1. Introduction
 2. Environment Setup
 3. Scenario: Campus Delivery with Bridge Congestion
 4. Part A: Problem Formulation (10 points)
 5. Part B: Bridge Congestion (10 points)
 6. Part C: Heuristic Function (10 points)
 7. Part D: A* Search (50 points)
 8. Challenge A: Best Heuristic Scale Factor (10 points)
 9. Challenge B: Multi-Stop Delivery (10 points)
 10. Grading Summary
 11. Academic Integrity
 12. Submission Instructions
-

1. Introduction

Purpose

In this COMP1037 coursework you will **formulate a search problem** and implement **A* search** to build a campus delivery agent. The agent must find optimal paths on a real campus map while respecting time-dependent constraints: bridges connecting regions separated by a lake become congested at certain hours, and the agent must plan routes that account for these extra costs.

Learning Outcomes

By completing this assignment, you will:

1. **Formulate** a search problem by defining state space, actions, goals, and path costs
2. **Implement** the A* search algorithm using a sorted frontier list
3. **Design** an admissible heuristic function (Euclidean distance)
4. **Handle** real-world constraints (bridge congestion that varies by time of day)
5. **Analyze** the relationship between heuristic strength and optimality
6. **Optimize** multi-stop delivery routing across permutations

Connection to Labs

This coursework directly builds on your lab experience:

Lab	Topic	Coursework Connection
Lab 3	Problem formulation, BFS, DFS	Part A and B: formulate the campus problem using the same patterns
Lab 4	UCS, Greedy, A* search, heuristics, sorted frontier	Parts C and D: implement heuristic and A* for campus navigation
Lab 5	Game search, algorithm evaluation	Challenges A and B: analyze heuristic scaling and multi-stop optimization

2. Environment Setup

Requirements

- **Python 3.9+** (same as the lab Jupyter environment)
- **Standard library only:** `json`, `math`, `itertools` -- no external packages needed
- No installation of `numpy`, `networkx`, `pygame`, or any third-party library is required

Files Provided

File	Purpose
<code>campus_delivery.py</code>	Your submission template. Contains function stubs, helper functions, and built-in local tests at the bottom.
<code>campus_map.json</code>	Campus data: 30 buildings, 92 directed edges, 3 bridges with congestion schedules. Do not modify.

File	Purpose
<code>visualize_campus_map.py</code>	Optional visualization utility. Requires <code>matplotlib</code> and <code>numpy</code> (already installed in the lab environment). Not needed for your submission.

Running Locally

Option 1: Terminal

```
python campus_delivery.py
```

SHELL

Option 2: Jupyter Notebook (recommended -- same environment as labs)

Place `campus_delivery.py` and `campus_map.json` in the **same folder** as your notebook. Then in a notebook cell:

```
%run campus_delivery.py
```

PYTHON

This runs the entire file including the built-in tests at the bottom. Output appears directly in the notebook.

If you modify `campus_delivery.py` externally and want to rerun, just execute the `%run` cell again -- it reloads the latest version automatically.

Alternatively, you can copy the code from `campus_delivery.py` into notebook cells and edit directly there. When you are ready to submit, copy the code back into `campus_delivery.py`.

The built-in tests at the bottom of `campus_delivery.py` are provided to help you verify your function signatures and understand expected input/output. They are **not** the grading tests. Your implementation should run within a few seconds on these built-in tests. If it takes significantly longer, you likely have an infinite loop.

We have **dedicated test cases for grading** that will evaluate your implementation after the submission deadline. These grading tests are not provided to students.

Return Value Format Requirements

To avoid losing marks due to formatting issues, make sure your functions return the **exact types** expected. The grading tests check types, not just values.

Function	Return Type	Example
<code>make_campus_problem()</code>	<code>dict</code>	<code>{'initial': 'GATE_1', 'goal': 'PMB', ...}</code>
<code>is_goal()</code>	<code>bool</code>	<code>True</code> or <code>False</code>
<code>campus_get_actions()</code>	<code>list</code> of <code>tuple</code>	<code>[('IAMET', 'IAMET'), ('IEB', 'IEB')]</code>
<code>campus_action_cost()</code>	<code>int</code> or <code>float</code>	A number (edge distance + any congestion), or <code>math.inf</code>
<code>heuristic_euclidean()</code>	<code>float</code>	A non-negative number (0.0 at goal)
<code>heuristic_scaled()</code>	<code>float</code>	A non-negative number
<code>find_best_scale()</code>	<code>float</code>	One of the candidate values (not a string, not an integer)
<code>a_star_search()</code>	<code>dict</code> or <code>None</code>	Node dict with <code>'state'</code> , <code>'cost'</code> , <code>'parent'</code> , etc.
<code>multi_stop_delivery()</code>	<code>tuple</code> of (<code>list</code> , number)	<code>([path_locations], total_cost)</code>

Important notes:

- **Building names are case-sensitive.** Use exactly the names from `campus_map.json` (e.g., `'Bridge_South'` not `'bridge_south'`).
- **Actions must be tuples**, not lists: `('IAMET', 'IAMET')` not `['IAMET', 'IAMET']`.
- **Costs are compared with tolerance** (`places=1`), so minor floating-point differences are fine. Both `int` and `float` are accepted for costs.
- `multi_stop_delivery` **must return a tuple**, not a list: `(path, cost)` not `[path, cost]`.

What You Can and Cannot Modify

`campus_delivery.py` contains two types of code:

- **Helper functions (Part 1):** `make_node`, `reconstruct_path`, `expand`, `load_campus_map`, `validate_path`, `is_cycle` -- these are **provided and must not be modified**. The grading system depends on them. Altering, renaming, or deleting any of these functions will cause grading failures.
- **Your implementations (Parts 2--5):** the 9 functions marked with `# TODO` -- these are the only parts you should modify. Fill in your code where indicated.

Do not add new files, rename the file, or change function signatures (names, parameters, return types). You may add your own helper functions if needed, but do not remove or modify any existing code outside the `# TODO` sections.

What to Submit

Upload **only** `campus_delivery.py` via the Moodle submission link. The grading server already has the test suite and campus map data.

3. Scenario: Campus Delivery with Bridge Congestion

The Campus

The UNNC campus is divided by **Nottingham Lake** into two regions:

- **West Campus** (academic and administrative): 15 locations including GATE_1, TRENT, PMB, Library, Auditorium, PB, Hub, IAMET, IEB, ICC, DB, New_Audi, NICC, Academician_Park, and YANG_Fujia
- **East Campus** (residential and sports): 12 locations including GATE_2, GATE_3, GATE_4, Staff_Hotel, LA_Hotel, Villas, Student_Canteen, Student_Residences_A, Student_Residences_B, Health_Centre, Sports_Centre, and Staff_Apartments_West

The two regions are connected by **three bridge nodes**:

- **Bridge_North** -- connects the academic core (Library, ICC, TRENT) to the residential area (Staff_Apartments_West, Staff_Hotel)
- **Bridge_Middle** -- connects the PB/Library area to Sports_Centre and Student_Canteen
- **Bridge_South** -- connects PMB/YANG_Fujia to DB/New_Audi

The campus graph has **30 buildings** and **92 directed edges** (roads) with pre-computed walking distances.

The Delivery Problem

A delivery agent must find the **cheapest path** between any two buildings on campus. The cost of travelling along a road is its walking distance (stored in `campus_graph`). However, bridges introduce a complication: **congestion**.

Bridge Congestion

Bridges on campus experience heavy foot traffic at predictable times:

- **Class change hours (9, 10, 11, 14, 15, 16)**: Hundreds of students walk across Bridge_South to reach the DB teaching area.
- **Meal hours (12, 13, 18, 19)**: Students flood Bridge_North and Bridge_Middle heading to

and from the canteen area.

When the delivery agent enters a bridge node during a congested hour, an **extra penalty** is added to the base walking distance. This penalty represents the delay caused by navigating through crowds.

The congestion data is stored in `campus_map.json` under `campus_bridges`:

```
"Bridge_South": {  
  "congestion": {"9": 500, "10": 400, "11": 400, "14": 500, "15": 400, "16":  
400}  
}
```

Hours use **24-hour format** as integer strings: `"9"` = 9:00, `"14"` = 14:00, `"18"` = 18:00. The `time_of_day` parameter in `make_campus_problem()` is a 24-hour integer (e.g., `time_of_day=9` for 9:00, `time_of_day=14` for 14:00).

This means: entering Bridge_South at `time_of_day=9` costs `base_distance + 500` extra units. At `time_of_day=6` (not listed in the schedule), there is no penalty -- the cost is just the base distance.

Summary of congestion schedules:

Bridge	Congested Hours	Penalty Range	Reason
Bridge_South	9, 10, 11, 14, 15, 16	400--500	Class changes near DB
Bridge_North	12, 13, 18, 19	400--500	Meal times near canteen
Bridge_Middle	12, 13, 18, 19	400--500	Meal times near canteen

Important: Travel Time Is Ignored

The delivery agent's travel time within a single delivery is ignored. Congestion is determined solely by the `time_of_day` parameter passed to `make_campus_problem()`. You do not need to track elapsed time as the agent moves. The agent is treated as moving instantaneously; the `time_of_day` determines congestion costs for the entire route.

Effect on Routing

Congestion changes which route is optimal. For example, at off-peak hours the shortest path between two buildings may cross a particular bridge directly, but at peak hours the congestion penalty on that bridge may make an alternative route through a different bridge cheaper.

Your A* implementation must account for these time-dependent costs to find truly optimal paths.

4. Part A: Problem Formulation (10 points)

Implement four functions that define the campus delivery search problem.

Note: The fourth function, `campus_action_cost()`, is listed here as part of the problem formulation, but its congestion-related behavior is separately evaluated in Part B.

Function 1: `make_campus_problem()`

PYTHON

```
def make_campus_problem(start, goal, campus_graph, campus_coords,
                        campus_bridges, time_of_day=9):
    """
    Create a campus delivery search problem.

    Args:
        start (str): initial location (building name)
        goal (str): goal location (building name)
        campus_graph (dict): {location: {neighbor: distance, ...}, ...}
        campus_coords (dict): {location: (x, y), ...}
        campus_bridges (dict): bridge definitions with congestion schedules
        time_of_day (int): hour of day (0-23), affects bridge congestion costs

    Returns:
        dict: problem with keys 'initial', 'goal', 'graph', 'coords',
        'bridges', 'time_of_day'
    """
```

Function 2: `is_goal()`

PYTHON

```
def is_goal(problem, state):
    """Check if state is the goal. Returns True if state ==
    problem['goal']."""
```

Function 3: `campus_get_actions()`

```
def campus_get_actions(problem, state):
    """
    Return list of possible actions from current state.
    Each action is a tuple (neighbor, neighbor).
    Simply return all neighbors from the graph -- no filtering needed.

    Returns:
        list of tuples: [(neighbor, neighbor), ...]
    """
```

Function 4: `campus_action_cost()`

```
def campus_action_cost(problem, state, action):
    """
    Return cost of moving from state to action (the next location).
    Cost = base graph distance + congestion penalty (if entering a bridge
    node).
    Returns infinity if the move is invalid (no edge exists).
    """
```

What is evaluated: Correct problem dictionary structure, goal checking, action generation, and basic cost calculation for non-bridge edges.

5. Part B: Bridge Congestion (10 points)

Bridge congestion is tested through the behavior of `campus_action_cost()` (defined in Part A). You do not need to implement a separate function -- these tests verify that your cost function correctly handles congestion.

When the agent moves to a bridge node, the cost should include any applicable congestion penalty for the current `time_of_day`. When a bridge is not congested at the given hour, or when the destination is not a bridge, only the base distance applies. If no edge exists between two locations, the cost should be infinity.

The congestion data is stored in `problem['bridges']`. Each bridge has a `'congestion'` dictionary mapping hour strings to penalty values.

What is evaluated: Correct congestion penalty calculation at peak hours, correct zero penalty at off-peak hours, and correct handling of invalid moves.

6. Part C: Heuristic Function (10 points)

Implement the Euclidean distance heuristic.

PYTHON

```
def heuristic_euclidean(state, problem):  
    """  
    Admissible heuristic: straight-line (Euclidean) distance from state to  
    goal.  
    h(state) = sqrt((x2 - x1)^2 + (y2 - y1)^2)  
  
    Properties:  
    - h(goal) = 0  
    - h(state) >= 0 for all states  
    - Admissible: never overestimates true optimal cost  
    """
```

Why this is admissible: The straight-line distance is always less than or equal to the actual road distance, because roads cannot be shorter than a straight line. Since congestion can only add cost (never reduce it), Euclidean distance never overestimates.

What is evaluated: Correct value at the goal, admissibility across multiple states and routes, and non-negativity.

7. Part D: A* Search (50 points)

Implement the A* search algorithm. This follows the same structure as the A* search from **Lab 4** -- adapt it to use the campus-specific functions and the provided helpers.

PYTHON

```
def a_star_search(problem, heuristic_fn):  
    """  
    A* search: finds optimal path using  $f(n) = g(n) + h(n)$ .  
  
    Args:  
        problem (dict): problem formulation  
        heuristic_fn: function(state, problem) -> float  
  
    Returns:  
        dict or None: goal node (use reconstruct_path() to get the path)  
    """
```

What is evaluated: Optimal paths on various routes, correct cost computation, congestion-aware routing, and termination.

8. Challenge A: Best Heuristic Scale Factor (10 points)

Background

Recall that an admissible heuristic must **never overestimate** the true cost to the goal. The basic Euclidean heuristic (`scale = 1.0`) is admissible because straight-line distance is always shorter than any road path. But it is also very conservative -- on this campus, roads are significantly longer than straight lines.

What if you multiply the Euclidean distance by a **scale factor**?

```
heuristic_scaled(state, problem, scale) = scale * Euclidean_distance(state, goal)
```

A larger scale makes the heuristic more "informed" (closer to the true cost), which makes A* expand fewer nodes. But if the scale is too large, the heuristic will **overestimate** on some (state, goal) pairs, breaking admissibility.

Your Task

Find the **largest scale factor** from the candidate list that is still admissible.

You need to implement two functions:

Function 1: `heuristic_scaled()`

```
def heuristic_scaled(state, problem, scale=1.0):  
    """  
    Returns scale * Euclidean_distance(state, goal).  
    """
```

PYTHON

Function 2: `find_best_scale()`

```
def find_best_scale(problem):
    """
    Find the LARGEST scale factor from the candidate list that keeps
    heuristic_scaled admissible.

    Candidates: [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0]

    Use off-peak time (time_of_day=6) for computing true optimal costs.
    The delivery agent's travel time is ignored -- congestion is based
    solely on the time_of_day parameter, not accumulated travel time.

    A heuristic is admissible if  $h(n) \leq \text{true\_cost}(n \rightarrow \text{goal})$  for ALL nodes  $n$ .

    Test goals (from these routes):
        [('GATE_1', 'PMB'), ('GATE_1', 'GATE_2'), ('Library', 'Staff_Hotel'),
        ('Hub', 'Health_Centre'), ('TRENT', 'Sports_Centre')]

    These routes define which goals to check. For each goal, you must
    verify admissibility across ALL nodes in the graph, not just the
    start of each route.

    Returns:
        float: the largest admissible scale from the candidates list
    """
```

Key questions to think about:

- How can you compute the true optimal cost from any node to a goal? (You already have A* with `scale=1.0`.)
- Why must you check all nodes, not just the start?
- Why test from largest to smallest?

What is evaluated: Correct largest admissible scale factor (evaluated at off-peak time), and that A* with the returned scale produces optimal paths while A* with an inadmissible scale does not.

9. Challenge B: Multi-Stop Delivery (10 points)

Task

Given a start location and three delivery locations {B, C, D}, find the ordering that **minimizes total travel cost** for the round trip: start → first → second → third → start.

There are 6 possible orderings of 3 locations. For each ordering, solve 4 A* problems (one per segment), sum the costs, and return the ordering with the lowest total cost.

```
def multi_stop_delivery(problem, locations_to_visit, heuristic_fn):  
    """  
    Find optimal ordering to visit 3 locations and return to start.  
  
    Args:  
        problem (dict): problem formulation (problem['initial'] is start)  
        locations_to_visit (list): 3 locations, e.g., ['PMB', 'Library', 'DB']  
        heuristic_fn: heuristic function for A*  
  
    Returns:  
        tuple: (full_path, total_cost)  
        - full_path: list of all locations visited in order,  
          e.g., ['GATE_1', ..., 'PMB', ..., 'Library', ..., 'DB', ...,  
            'GATE_1']  
        - total_cost: sum of all segment costs  
    """
```

PYTHON

When concatenating path segments, avoid duplicating the shared endpoint. If segment 1 ends at 'PMB' and segment 2 starts at 'PMB', include 'PMB' only once.

Important: Congestion applies to multi-stop delivery. The `time_of_day` from the original problem must be passed to each sub-problem when calling A*. If the delivery departs during a congested hour, the congestion penalties affect every segment of the route. Do not ignore the `time_of_day` when creating sub-problems.

What is evaluated: Correct return format, path visiting all locations and returning to start, and optimal total cost across all permutations (with congestion).

10. Grading Summary

Your submission will be evaluated by 24 dedicated grading tests that run after the deadline closes. The grading tests are separate from the built-in tests in `campus_delivery.py`.

Component	Points	What Is Evaluated
Part A: Problem Formulation	10	Correct problem dict, goal checking, action generation
Part B: Bridge Congestion	10	Correct congestion cost calculation

Component	Points	What Is Evaluated
Part C: Heuristic	10	Admissible Euclidean heuristic
Part D: A* Search	50	Optimal paths on various routes with congestion
Challenge A: Best Scale	10	Correct best admissible scale factor
Challenge B: Multi-Stop	10	Optimal multi-stop delivery ordering
Total	100	

All grading is automated. There is no manual review.

11. Academic Integrity

Use of AI Tools

You **must not** use AI tools to:

- Generate complete function implementations and submit them as your own work
- Copy-paste AI output without understanding what it does
- Bypass the learning process by having AI do all the thinking

The standard: You must be able to **explain every line of code you submit**. If you cannot explain how your A* implementation works, or why your heuristic is admissible, then the work is not yours.

Collaboration

This is an **individual assignment**. You must complete it alone.

- Discussing high-level approaches is acceptable ("I used `is_cycle()` to avoid loops").
- Sharing code with other students is not acceptable, before or after submission.
- Copying from classmates, online solutions, or any other source is academic dishonesty.

Consequences

Violations of academic integrity will be reported to the university and may result in a zero for this coursework, course failure, or formal misconduct proceedings. When in doubt, ask.

12. Submission Instructions

What to Submit

- **File:** `campus_delivery.py` (your completed implementation)
- **Platform:** Submit via the **Moodle submission link**, which redirects to Gradescope for automated grading
- **Deadline:** 4pm, 11 May 2026 -- this is a hard deadline with no extensions

Important: Allow Extra Time for Submission

Most of you have not used Gradescope before. The Moodle submission link will redirect you to the Gradescope platform. **Allow extra time** for account setup, file upload, and any technical issues. Do not wait until the last hour to submit.

How Grading Works

1. Complete your implementations in `campus_delivery.py`
2. Test locally by running `python campus_delivery.py` to verify your functions work with the built-in tests
3. Submit `campus_delivery.py` via the Moodle link before the deadline
4. **After the deadline**, the grading system runs the dedicated test suite against your submission
5. You will receive your score and results after grading is complete

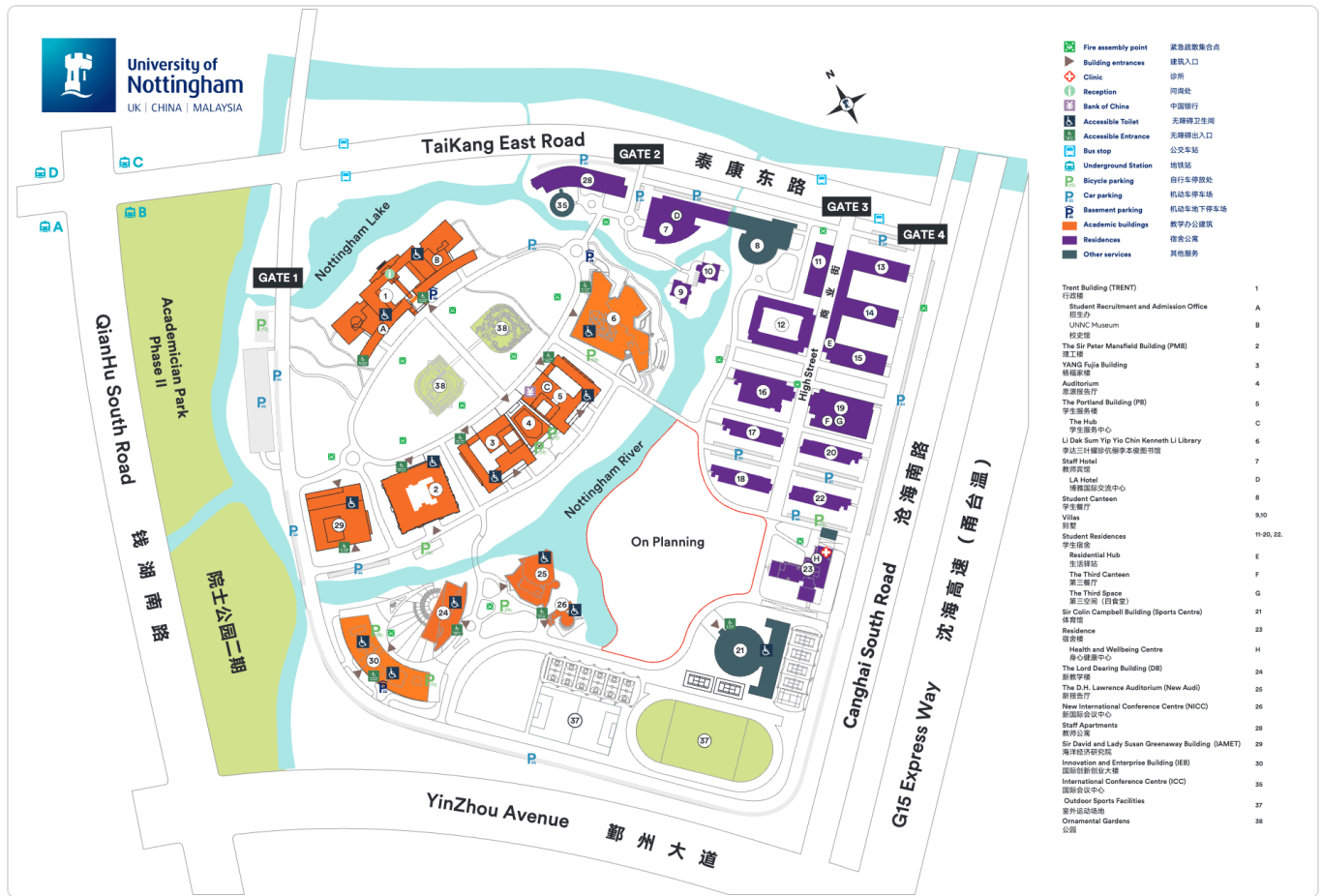
There is no resubmit-and-fix process. You cannot use the grading system as a testing tool. Make sure your code works correctly with the built-in tests before submitting. The built-in tests in `campus_delivery.py` cover all key function signatures and demonstrate expected input/output formats -- use them thoroughly.

Resources

- **Lab 3:** Problem formulation, `make_node()`, `reconstruct_path()`, `expand()`, BFS/DFS
- **Lab 4:** A* implementation, sorted frontier list, heuristic design, UCS/Greedy
- **Python** `itertools.permutations()`: <https://docs.python.org/3/library/itertools.html>
- **Python** `math.sqrt()`: <https://docs.python.org/3/library/math.html>



University of
Nottingham
UK | CHINA | MALAYSIA



- Fire assembly point 紧急疏散集合点
- Building entrances 建筑入口
- Clinic 诊所
- Reception 接待处
- Bank of China 中国银行
- Accessible Toilet 无障碍卫生间
- Accessible Entrance 无障碍出入口
- Bus stop 公交车站
- Underground Station 地铁站
- Bicycle parking 自行车停放处
- Car parking 机动车停车场
- Basement parking 机动车地下停车场
- Academic buildings 教学办公建筑
- Residences 宿舍公寓
- Other services 其他服务

- Trent Building (Trent) 行政楼 1
- Student Recruitment and Admission Office 招生办 A
- UNHC Museum 纪念馆 B
- The Sir Peter Mansfield Building (PMB) 2
- YANGLI Building 杨福康楼 3
- Auditorium 思源报告厅 4
- The Portland Building (PR) 5
- The Hub 学生服务中心 C
- Li Dak Sum Yip Yio Chin Kenneth Li Library 李达三叶耀珍叶耀李本殿图书馆 6
- Staff Hotel 教工酒店 7
- LA Hotel 澜海国际交流中心 D
- Student Canteen 学生餐厅 8
- Villages 别墅 9, 10
- Student Residences 学生宿舍 11-20, 22
- Residential Hub 生活驿站 E
- The Third Canteen 第三餐厅 F
- The Third Space 第三空间 (自习室) G
- Sir Colin Campbell Building (Sports Centre) 体育馆 21
- Residence 宿舍楼 23
- Health and Wellbeing Centre 身心健康中心 H
- The Lord Dearing Building (DB) 新教学楼 24
- The D.H. Lawrence Auditorium (New Aul) 新报告厅 25
- New International Conference Centre (NICC) 新国际会议中心 26
- Staff Apartments 教师公寓 28
- Sir David and Lady Susan Greenaway Building (IAMET) 戴维和苏珊格林威建筑 29
- Innovation and Enterprise Building (IEB) 国际创新创业大楼 30
- International Conference Centre (ICC) 国际会议中心 35
- Outdoor Sports Facilities 室外运动场地 37
- Ornamental Gardens 公园 38

